

Programming in Flutter

Animations

What are animations?

It is the motion on the screen - many still images shown in quick succession

Why animations?

Functional

- Meaningful transitions
- Provide visual feedback
- Show system status
- Help user get started (e.g. by displaying animation of desired interaction)

Delightful

- To demonstrate polish of the app
- To bring personality and humanity to the app
- To entertain users

Implicit vs Explicit Animations

Implicit Animations

Implicit animations

- AnimatedContainer
- AnimatedAlign
- AnimatedPositioned
- AnimatedPositionedDirectional
- AnimatedOpacity
- AnimatedPadding
- AnimatedTheme
- AnimatedCrossFade

And many more... See here: <https://docs.flutter.dev/ui/widgets/animation>

Implicit Animations

Do not require a controller!



```
class _PageState extends State<Page> {  
  double _opacity = 1.0;  
  
  @override  
  Widget build(BuildContext context) {  
    return AnimatedOpacity(  
      opacity: _opacity,  
      duration: const Duration(milliseconds: 300),  
      child: const Text('This will animate'),  
    );  
  }  
}
```

Explicit Animations

AnimationController

We control the ~~traffic~~ animation



AnimationController

We are responsible for initializing it, but also for the disposal.

```
class _PageState extends State<Page> with SingleTickerProviderStateMixin {  
  late final AnimationController _animationController;  
  
  @override  
  void initState() {  
    super.initState();  
    _animationController = AnimationController(vsync: this);  
  }  
  
  @override  
  void dispose() {  
    _animationController.dispose();  
    super.dispose();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return ...;  
  }  
}
```

AnimationController

With great responsibility,
comes great power.

```
_controller.isAnimating;  
_controller.lastElapsedDuration;  
_controller.status;  
_controller.value;  
_controller.velocity;  
_controller.view;  
  
_controller.forward(from: 0);  
_controller.reverse(from: 1);  
_controller.repeat();  
_controller.stop();  
_controller.reset();  
  
_controller.addListener(() {  
  print('AnimationController: ${_controller.value}');  
});  
  
_controller.addStatusListener(  
  (status) => print('AnimationController: $status'),  
);  
  
// and many more
```

AnimationController

It can be ugly,
but its effects are beautiful.



```
class _AnimationControllerDemoState extends State<_AnimationControllerDemo>
  with SingleTickerProviderStateMixin {
  late final AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 2),
    );

    _controller.repeat(reverse: true);
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Transform.scale(
      scale: _controller.value,
      child: const Text('hello'),
    );
  }
}
```

AnimationController

It is not all about *AnimationController*
Animation is a close friend as well.

```
class _MyPageState extends State<MyPage>
    with SingleTickerProviderStateMixin {
    late final AnimationController _animationController;
    late final Animation<double> _animation;

    @override
    void initState() {
        super.initState();

        _animationController = AnimationController(
            vsync: this,
            duration: const Duration(seconds: 2),
        );

        _animation = CurvedAnimation(
            parent: _animationController,
            curve: Curves.elasticOut,
        );

        _animationController.repeat();
    }

    @override
    Widget build(BuildContext context) {
        return AnimatedBuilder(
            animation: _animationController,
            builder: (context, child) => Transform.scale(
                scale: _animation.value,
                child: child,
            ),
            child: ...
        );
    }
}
```

Animation curves

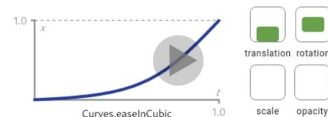
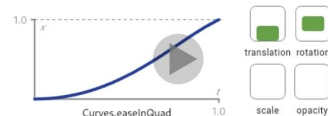
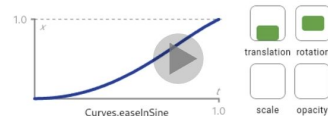
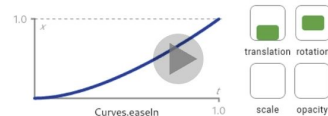
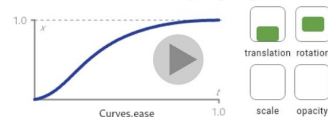
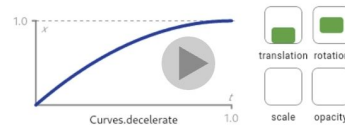
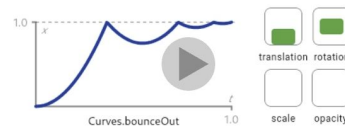
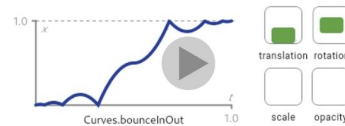
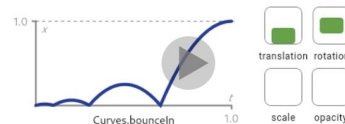
What is a curve?

- A function that maps **time progress** ($t \in 0..1$) $\rightarrow$ **value progress**
- It does **not change the animation duration**, only its **dynamics**
- The same animation with a different curve

= a completely different **UX feel**

Curves class abstract final

A collection of common animation curves.



Animation Springs

What is a spring?

- A **physical spring model**, not a mathematical curve
- Simulates **object motion in real time**
- Reacts to:
 - animation interruption
 - target change
 - current velocity



Animation Springs

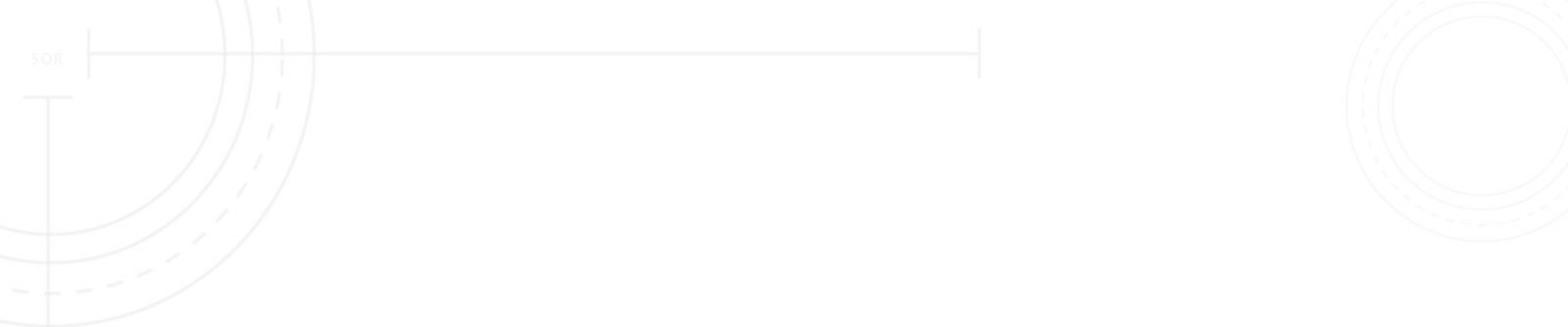
Spring APIs

- *SpringDescription*
 - *mass, stiffness, damping*
- *SpringSimulation*
 - *(spring, start, end, velocity)*
- *AnimationController.animateWith(simulation)*
 - controller does not need *duration*
 - animation duration is deducted from the physics

Animation Springs

Connects to what we already know!

```
final spring = SpringDescription(  
    mass: 1.0,  
    stiffness: 500.0,  
    damping: 25.0,  
);  
  
final simulation = SpringSimulation(  
    spring,  
    0.0,  
    1.0,  
    0.0,  
);  
  
_controller.animateWith(simulation);
```



Curves vs Springs

Curves vs Springs

Curves

- Mathematical function
- Predefined behavior
- Fixed duration
- No concept of velocity
- No reaction to interruption
- Deterministic

Typowe use-case'y:

- fade in / out
- icon scaling
- screen transitions
- micro-interactions
- loading indicators

Springs

- Physical model (motion simulation)
- No fixed duration
- Has velocity
- Reacts to target changes
- Reacts to interruption
- Non-deterministic (input-dependent)

Typowe use-case'y:

- drag & swipe
- bottom sheets
- cards (Tinder-style)
- fling after release

What can be animated?



Tweens

Tween

What is Tween?

- Maps values *begin* -> *end*
- Does not animate time - only maps the values from the controller
- Works with various types.

```
final animation = Tween<double>(begin: 0.5, end: 1.0)  
  .animate(controller);
```

Tweens

Some other examples

```
animation = ColorTween(  
  begin: Colors.yellow,  
  end: Colors.orange,  
)  
.animate(_animationController);
```

```
animation = SizeTween(  
  begin: const Size(200, 50),  
  end: const Size(50, 200),  
)  
.animate(_animationController);
```

```
animation = BorderRadiusTween(  
  begin: BorderRadius.circular(0),  
  end: BorderRadius.circular(80),  
)  
.animate(_animationController);
```

Tweens

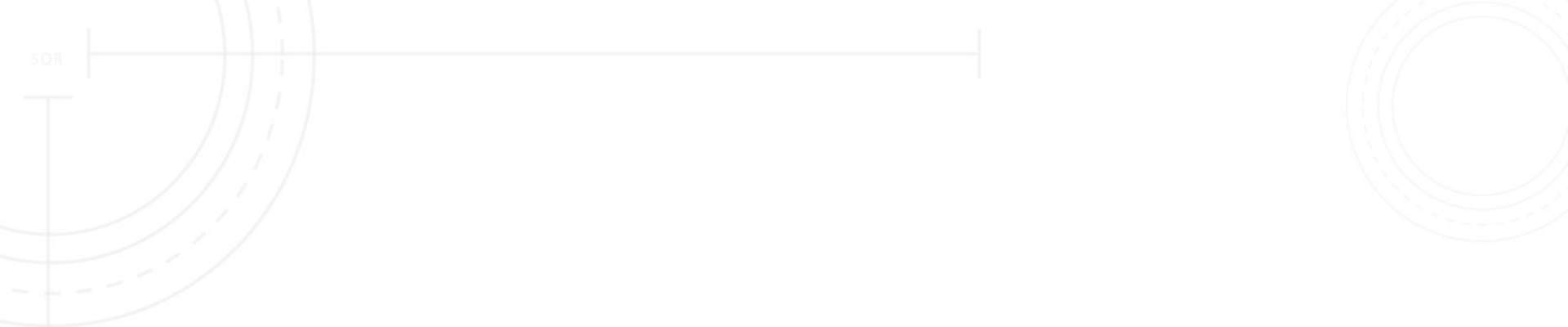
Multiple animations can be bounded to the same controller.

```
_borderAnimation = BorderRadiusTween(  
  begin: BorderRadius.circular(0),  
  end: BorderRadius.circular(80),  
)  
.animate(CurvedAnimation(  
  parent: _animationController,  
  curve: const Interval(0, 0.4),  
));  
  
_sizeAnimation = SizeTween(  
  begin: const Size(200, 50),  
  end: const Size(50, 200),  
)  
.animate(CurvedAnimation(  
  parent: _animationController,  
  curve: const Interval(0.3, 0.7),  
));  
  
_colorAnimation = ColorTween(  
  begin: Colors.yellow,  
  end: Colors.red,  
)  
.animate(CurvedAnimation(  
  parent: _animationController,  
  curve: const Interval(0.7, 1.0),  
));
```


Tweens

TweenAnimationBuilder can come in handy and save us some boilerplate.

```
class _PageState extends State<Page> {  
  
  @override  
  Widget build(BuildContext context) {  
    return TweenAnimationBuilder(  
      tween: IntTween(begin: 0, end: 100),  
      duration: const Duration(seconds: 1),  
      builder: (context, value, child) =>  
        Text(value.toString()),  
    );  
  }  
}
```



Can we make it even easier?

of course!

Transitions



RelativePositionedTransition

Animated version of Positioned which transitions the child's position based on the value of rect relative to a bounding box with the specified size.



RotationTransition

Animates the rotation of a widget.



ScaleTransition

Animates the scale of transformed widget.



SizeTransition

Animates its own size and clips and aligns the child.



SlideTransition

Animates the position of a widget relative to its normal position.



SliverFadeTransition

Animates the opacity of a sliver widget.

Find more widgets in the [widget catalog](#).

Transitions

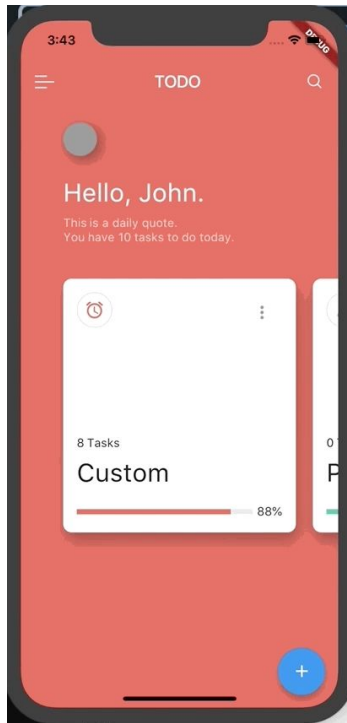
Just look for proper Transition widget :)

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: RotationTransition(
        turns: _animationController,
        child: Container(
          color: Colors.red,
          width: 300,
          height: 300,
        ),
      ),
    ),
  );
}
```

Some other tricks...

Heroes

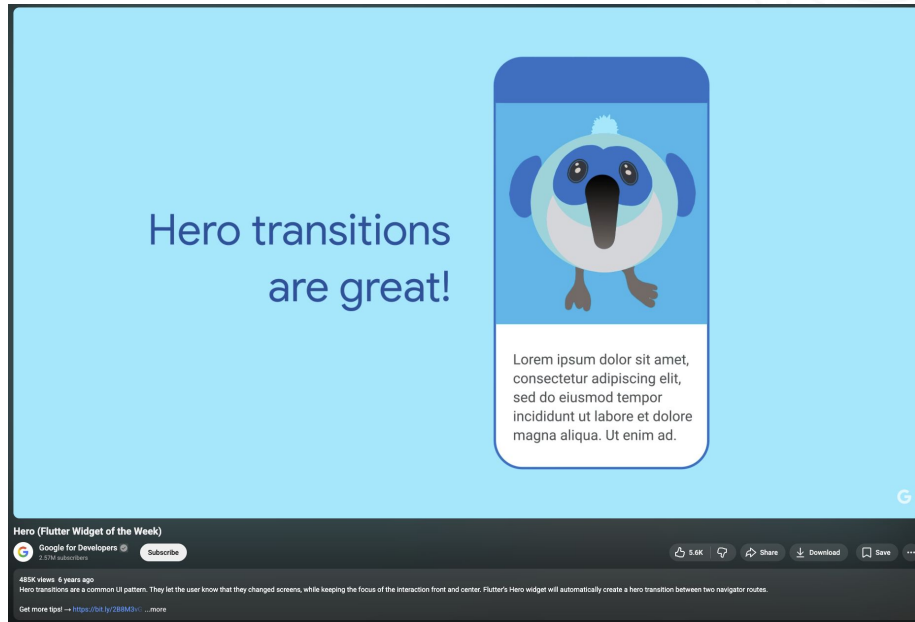
Heroes



<https://github.com/flutter/flutter/issues/17627>

Heroes

The hero refers to the widget that flies between screens



<https://www.youtube.com/watch?v=Be9UH1kXFDw>

Heroes

Two hero widgets with the same tag allow for good looking transition of the widget from the source route to the destination route.

```
class PhotoHero extends StatelessWidget {
  const PhotoHero({
    super.key,
    required this.photo,
    this.onTap,
    required this.width,
  });

  final String photo;
  final VoidCallback? onTap;
  final double width;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Hero(
        tag: photo,
        child: Material(
          color: Colors.transparent,
          child: InkWell(
            onTap: onTap,
            child: Image.asset(
              photo,
              fit: BoxFit.contain,
            ),
          ),
        ),
      ),
    );
  }
}
```

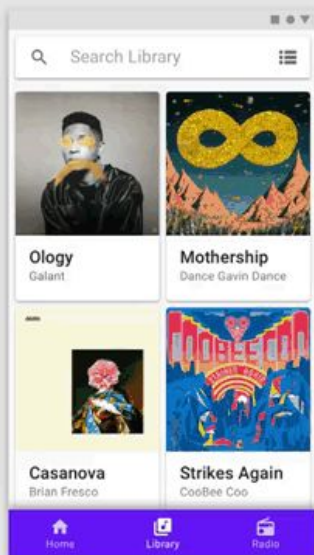
```
class HeroAnimation extends StatelessWidget {
  const HeroAnimation({super.key});

  Widget build(BuildContext context) {
    timeDilation = 5.0;

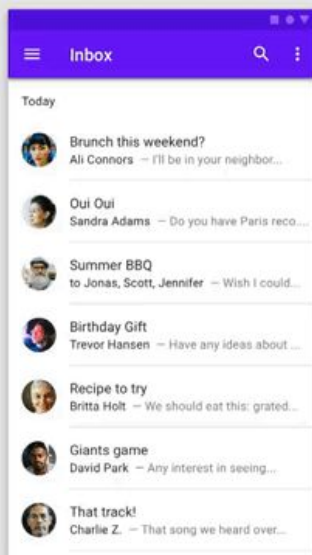
    return Scaffold(
      appBar: AppBar(
        title: const Text('Basic Hero Animation'),
      ),
      body: Center(
        child: PhotoHero(
          photo: 'images/flippers-alpha.png',
          width: 300.0,
          onTap: () {
            Navigator.of(context).push(MaterialPageRoute<void>({
              builder: (context) {
                return Scaffold(
                  appBar: AppBar(
                    title: const Text('Flippers Page'),
                  ),
                  body: Container(
                    color: Colors.lightBlueAccent,
                    padding: const EdgeInsets.all(16),
                    alignment: Alignment.topLeft,
                    child: PhotoHero(
                      photo: 'images/flippers-alpha.png',
                      width: 300.0,
                      onTap: () {
                        Navigator.of(context).pop();
                      },
                    ),
                  ),
                );
              },
            ));
          },
        ),
      ),
    );
  }
}
```

Animations package

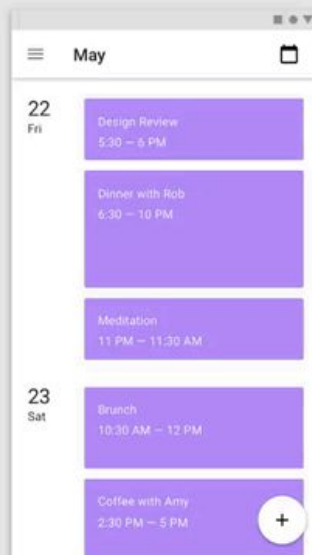
Animations package



1



2



3



4

Animations package

High quality pre-built Animations
for Flutter!

```
class _OpenContainer extends StatelessWidget {  
  const _OpenContainer();  
  
  @override  
  Widget build(BuildContext context){  
    return OpenContainer(  
      transitionBuilder: Duration(milliseconds: 1000),  
      closedBuilder: (_, __) => const Center(  
        child: ListTile(title: Text('Item')),  
      ),  
      openBuilder: (_, __) => Scaffold(  
        appBar: AppBar(title: const Text('Page 2')),  
        body: const Center(child: Text('page2')),  
      ),  
    );  
  }  
}
```

Rive

Rive

Build the animation within
the editor and just
load it in the Flutter app!



<https://medium.com/@RotenKiwi/rive-for-flutter-animations-a99bfdd8f6cc>

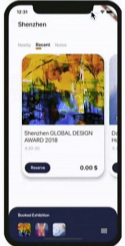
Next steps

Next steps

Is my animation more like a drawing?



@HelloJCToon (Rive)



@marcin_szalek

Flutter Europe

Emily Fortuna & Andrew Fitz Gibbon
Animations in Flutter Done Right

Animations in Flutter Done Right - Emily Fortuna & Andrew Fitz Gibbon | Flutter Europe

Flutter Europe 17.4K subscribers

313 views 5 years ago

Flutter has a wealth of animation widgets - so many it can be overwhelming knowing which one to choose. We'll walk through the process of choosing the right animation widget for your use case and how to incorporate animations into your app to make them "pop" rather than "stop."

Speakers: Emily Fortuna (@bouncingghost) & Andrew Fitz Gibbon (@fitz) ...more

<https://www.youtube.com/watch?v=pq791sJtWBQ>

„People use only 10% of Flutter”

me

Flutter Europe

Marcin Szałek
Implementing complex UI with Flutter

Implementing complex UI with Flutter - Marcin Szałek | Flutter Europe

Flutter Europe 17.4K subscribers

10K views 5 years ago

Flutter was created to make beautiful apps but do we really know how to use it? When coming across complex designs we might think that they are impossible to implement or too difficult to handle without weeks of development. In this talk, I will show you how you can approach complex designs and translate them into widgets. We will look closely at some mobile designs, break them down and figure out how to code them.

<https://www.youtube.com/watch?v=FCyoHclCqc8>

Next steps

<https://docs.flutter.dev/ui/animations>

Introduction to animations

UI > Animations

Well-designed animations make a UI feel more intuitive, contribute to the slick look and feel of a polished app, and improve the user experience. Flutter's animation support makes it easy to implement a variety of animation types. Many widgets, especially [Material widgets](#), come with the standard motion effects defined in their design spec, but it's also possible to customize these effects.

Choosing an approach

There are different approaches you can take when creating animations in Flutter. Which approach is right for you? To help you decide, check out the video, [How to choose which Flutter Animation Widget is right for you?](#) (Also published as a [companion article](#).)

